Laboratorio #3 — Investigación

Sistema de inventario para una aduana: base de datos relacional y agente de IA con n8n

Integrantes: _____ · Asignatura: Bases de Datos · Fecha: 23/10/2026

A. Bases de datos y SQL

1. Base de datos y SGBD

Una **base de datos** es un conjunto organizado de datos relacionados entre sí, guardados de forma que se puedan consultar, actualizar y proteger con facilidad. En nuestro caso, la base de datos de la aduana guarda las declaraciones, los productos, las bodegas y las inspecciones; hoy esa información está repartida en hojas de cálculo, y eso provoca duplicados y mercancía "perdida".

Un **Sistema Gestor de Bases de Datos (SGBD)** es el programa que administra la base de datos: guarda los datos en disco, entiende el lenguaje SQL, controla quién puede ver o modificar cada tabla, evita que dos usuarios dañen los datos al mismo tiempo (concurrencia) y permite recuperar la información si algo falla (respaldos y transacciones). Ejemplos: PostgreSQL, MySQL, SQLite, Oracle y SQL Server.

2. Bases de datos relacionales vs. no relacionales (NoSQL)

Aspecto	Relacional (SQL)	No relacional (NoSQL)
Estructura	Tablas con filas y columnas y un esquema fijo	Documentos JSON, pares clave-valor, grafos o columnas; esquema flexible
Relaciones	Llaves primarias y foráneas, JOIN	Normalmente se duplican datos o se anidan documentos
Consistencia	Transacciones ACID (todo o nada)	Muchas priorizan velocidad y escalado horizontal
Ejemplos	PostgreSQL, MySQL, SQLite	MongoDB, Redis, Cassandra, Neo4j
Cuándo usarla	Datos muy relacionados y que deben ser exactos (dinero, inventario)	Datos cambiantes o masivos (redes sociales, sesiones, caché)

Para una aduana conviene una base **relacional**: una declaración se relaciona con un importador, un agente, un país, productos, contenedores e impuestos, y los montos deben cuadrar siempre.

3. Tabla, registro, campo y tipos de datos

- **Tabla:** representa una entidad, por ejemplo `bodega`.
- **Registro (fila):** un elemento concreto, por ejemplo la "Bodega 2 – Puerto Quetzal Patio B".
- **Campo (columna):** una característica que tienen todos los registros, por ejemplo `capacidad_m3`.
- **Tipos de datos comunes:** `INT` e `INT AUTO_INCREMENT` (enteros y autoincrementales), `DECIMAL(12,2)` (dinero y cantidades exactas, sin errores de redondeo), `VARCHAR(n)` y `TEXT` (texto), `CHAR(2)` (texto de largo fijo, como el código ISO "CN"), `DATE` y `DATETIME/TIMESTAMP` (fechas y fecha-hora) y `BOOLEAN` (verdadero/falso).

4. Llave primaria, foránea y compuesta

- **Llave primaria (PK):** columna que identifica de forma única cada fila y no puede ser nula. Ej.: `id_declaracion`.

- **Llave foránea (FK):** columna que apunta a la PK de otra tabla y así crea la relación. Ej.: `declaracion.id_pais_origen` → `pais.id_pais`. El SGBD no deja registrar una declaración de un país que no existe.
- **Llave compuesta:** PK formada por dos o más columnas. En `detalle_declaracion` la PK es `(id_declaracion, id_producto)`: un mismo producto no puede repetirse dentro de la misma declaración.

5. Tipos de relaciones y cómo se implementan

Tipo	Ejemplo en la aduana	Implementación
1:1	Una declaración tiene **una** liquidación de impuestos	FK con restricción `UNIQUE` (`liquidacion.id_declaracion UNIQUE`)
1:N	Un país es origen de muchas declaraciones	FK en la tabla del lado "muchos" (`declaracion.id_pais_origen`)
N:M	Una declaración tiene muchos productos y un producto aparece en muchas declaraciones	Tabla intermedia `detalle_declaracion` con dos FK
Recursiva	Un funcionario supervisa a otros funcionarios	FK a la misma tabla (`funcionario.id_supervisor`)

6. Integridad referencial y restricciones

La **integridad referencial** garantiza que toda FK apunte a un registro que existe: no puede haber un contenedor de una declaración inexistente, y no se puede borrar un país que tiene declaraciones. Las **restricciones** son reglas que el SGBD verifica en cada INSERT o UPDATE:

- `NOT NULL`: el campo es obligatorio (toda declaración debe tener fecha).
- `UNIQUE`: no se repite (el número de DUA o el NIT).
- `CHECK`: condición lógica (`porcentaje_arancel BETWEEN 0 AND 100`, `regimen IN ('Importacion','Exportacion','Transito','Deposito')`).
- `DEFAULT`: valor por omisión (una declaración nueva queda en estado `'Registrada'`).

7. DDL, DML y sentencias básicas

- **DDL (Data Definition Language)** define la estructura: `CREATE`, `ALTER`, `DROP`.
- **DML (Data Manipulation Language)** trabaja con los datos: `INSERT`, `SELECT`, `UPDATE`, `DELETE`.

```
CREATE TABLE pais (id_pais INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(60) NOT NULL UNIQUE);
INSERT INTO pais (nombre) VALUES ('China');
SELECT nombre FROM pais WHERE nombre LIKE 'c%';
UPDATE declaracion SET estado = 'Liberada' WHERE numero = 'DUA-2026-0013';
DELETE FROM contenedor WHERE id_contenedor = 40;
-- JOIN: unir tablas por sus llaves
SELECT d.numero, p.nombre AS pais
FROM declaracion d JOIN pais p ON p.id_pais = d.id_pais_origen;
```

`JOIN` (o `INNER JOIN`) devuelve solo las filas que coinciden en ambas tablas; `LEFT JOIN` devuelve además las filas de la tabla izquierda sin pareja (en nuestra consulta Q10 aparecen funcionarios con 0 inspecciones gracias a `LEFT JOIN`).

8. Normalización (1FN, 2FN y 3FN)

Normalizar es organizar las tablas para **evitar datos repetidos y contradicciones**.

- **1FN:** cada celda guarda un solo valor y no hay grupos repetidos. Mal: una columna `productos = "laptops, celulares"`. Bien: una fila por producto en `detalle_declaracion`.
- **2FN:** está en 1FN y, si la PK es compuesta, cada columna depende de **toda** la llave. En `detalle_declaracion (id_declaracion, id_producto)` la `cantidad` depende de ambos; la `descripcion` del producto depende solo de `id_producto`, por eso vive en `producto`.
- **3FN:** está en 2FN y no hay dependencias transitivas (una columna no clave que depende de otra columna no clave). El `porcentaje_arancel` depende de la partida, no del producto; por eso se separó en `partida_arancelaria` y el producto solo guarda `id_partida`. Si cambia el arancel, se actualiza en un solo lugar.

9. Diagrama Entidad-Relación y notaciones

El **DER** es el "plano" de la base de datos: muestra entidades (tablas), atributos y relaciones con su cardinalidad antes de escribir SQL.

- **Notación de Chen:** entidades en rectángulos, atributos en óvalos y relaciones en rombos, con la cardinalidad (1, N, M) escrita sobre las líneas. Es muy didáctica pero ocupa mucho espacio.
- **Notación pata de gallo (Crow's Foot):** cada entidad es una caja con sus atributos; los extremos de la línea indican la cardinalidad: `||` uno y solo uno, `o|` cero o uno, `|<` uno o muchos y `o<` cero o muchos (la "pata de gallo"). Es la que usan herramientas como dbdiagram.io y MySQL Workbench, y la que usamos en nuestro diagrama.

B. Gestores de bases de datos

Criterio	SQLite	MySQL	PostgreSQL
Arquitectura	Archivo único, sin servidor	Cliente-servidor	Cliente-servidor
Instalación	La más simple	Fácil (XAMPP, Docker)	Fácil; gratis en la nube con Supabase o Neon
Conexión con n8n	Sin nodo nativo para el agente (requiere trucos o migrar)	Nodo nativo MySQL	Nodo nativo Postgres (también como herramienta del agente)
Usuarios y permisos	No tiene usuarios	Sí (`CREATE USER` + `GRANT SELECT`)	Sí, muy completos: roles, solo lectura, Row Level Security
Restricciones `CHECK`	Sí	Sí (desde 8.0.16)	Sí, desde siempre
Extras	—	Muy usado en web (WordPress, PHP); columnas generadas, `REGEXP_LIKE`, tipo JSON	`FILTER`, Row Level Security, tipos avanzados

Elegimos MySQL 8 porque: (1) n8n tiene un nodo nativo de MySQL que el agente puede usar directamente como herramienta; (2) permite crear el usuario `agente_lectura` con `GRANT SELECT` únicamente sobre la base `aduana`, que es justo lo que pide el laboratorio por seguridad; (3) ya lo teníamos instalado junto con **MySQL Workbench**, que sirve para ejecutar el script y ver las tablas; (4) es el gestor más usado en aplicaciones web, y desde la versión 8 valida las restricciones `CHECK` y permite columnas calculadas (usamos una para `total_pagar`). Usamos el juego de caracteres `utf8mb4` con la intercalación `utf8mb4_0900_ai_ci`, que guarda tildes y ñ y hace que las búsquedas no distingan mayúsculas ni tildes ('mexico' encuentra 'México'), algo muy útil para el agente. El script se ejecuta sin errores de principio a fin (incluso dos veces seguidas, porque primero borra las tablas si existen). También preparamos una versión para PostgreSQL por si se quiere usar Supabase.

C. n8n y agentes de IA

1. **n8n** es una herramienta de automatización de flujos de trabajo (*workflows*) de código abierto. Un workflow es una secuencia de pasos que se ejecuta sola cuando ocurre algo: "cuando llegue un mensaje → consulta la base de datos → responde". Se arma visualmente conectando cajas, sin tener que programar todo.
2. **Nodo:** cada caja del flujo; hace una tarea (consultar MySQL, llamar a un modelo de IA). **Trigger (disparador):** el nodo que inicia el flujo (en nuestro caso el *Chat Trigger*, que abre una ventana de chat). **Credencial:** los datos de acceso a un servicio (usuario y clave de la base de datos, API key de Gemini) que n8n guarda cifrados aparte del flujo; por eso al exportar el JSON no se incluyen contraseñas.
3. **Agente de IA vs. chatbot normal:** un chatbot tradicional responde con reglas o respuestas fijas ("si escribe *horario* → responde 8 a 5"). Un **agente** usa un modelo de lenguaje para **razonar**, decide qué herramienta usar, la ejecuta, revisa el resultado y vuelve a intentar si falló. Nuestro agente traduce "¿qué mercancías de China siguen retenidas en la bodega 2?" a una consulta SQL, la ejecuta y redacta la respuesta.
4. **LLM (Large Language Model):** modelo entrenado con enormes cantidades de texto que predice y genera lenguaje (Gemini, GPT, Claude). Las **herramientas (tools)** son capacidades que le damos al agente para actuar fuera del modelo: consultar una base de datos, buscar en la web, enviar un correo. Sin herramientas, el LLM solo "inventaría" la respuesta; con la herramienta MySQL responde con datos reales.
5. **Consulta segura:** (a) el agente se conecta con un usuario **de solo lectura** (`agente_lectura`) que solo tiene `GRANT SELECT ON aduana.*` y como máximo 5 conexiones simultáneas; probamos que un `DELETE`, `UPDATE`, `INSERT` o `DROP TABLE` con ese usuario es rechazado por MySQL (*ERROR 1142: command denied*); (b) el usuario solo puede conectarse desde la misma computadora (`'agente_lectura'@'localhost'`); (c) el *system prompt* le prohíbe modificar datos, pero **la seguridad real está en la base de datos**, porque un usuario malicioso podría intentar engañar al modelo (*prompt injection*); (d) las claves se guardan en credenciales de n8n, nunca en el flujo.

D. El dominio aduanero

- **Aduana:** institución pública que controla la entrada y salida de mercancías del país, cobra los impuestos de comercio exterior y evita el contrabando y el ingreso de productos prohibidos o peligrosos.
- **Declaración aduanera (DUA):** documento oficial en el que el importador o exportador, normalmente mediante su agente aduanal, declara qué mercancía mueve, su cantidad, valor, origen y el régimen al que se acoge. Es la entidad central de nuestro modelo.
- **Partida arancelaria (Sistema Armonizado):** código internacional de la Organización Mundial de Aduanas que clasifica todas las mercancías. Los primeros 6 dígitos son iguales en todo el mundo (ej. `8471.30` = computadoras portátiles; `8517.13` = teléfonos inteligentes) y cada país agrega dígitos propios. El **arancel** es el impuesto que se paga por importar, generalmente un porcentaje del valor CIF (costo + seguro + flete) que depende de la partida; además se cobra IVA.
- **Regímenes básicos: importación** (la mercancía entra para quedarse y paga impuestos); **exportación** (sale del país, normalmente sin aranceles); **tránsito** (cruza el país hacia otro destino sin pagar impuestos, bajo control); **depósito** (se guarda en un almacén fiscal sin pagar impuestos hasta que se decida importarla o reexportarla).
- **Agente aduanal:** profesional con licencia autorizado para tramitar declaraciones en nombre de importadores y exportadores; clasifica la mercancía, calcula impuestos y responde ante la aduana por la información declarada.

E. Resumen del diseño realizado

- **14 entidades** (mínimo pedido: 8): país, importador, agente aduanal, partida arancelaria, producto, funcionario, bodega, declaración, detalle de declaración, contenedor, inventario, movimiento de inventario, inspección y liquidación.
- Relaciones **1:N** (país–declaración, partida–producto, declaración–contenedor), **N:M** resuelta con `detalle_declaracion`, **1:1** (declaración–liquidación) y **recursiva** (funcionario–supervisor).
- Datos de prueba coherentes: 25 declaraciones de agosto a octubre de 2026, 18 países reales, partidas del Sistema Armonizado reales, arancel e IVA (12 %) calculados a partir del valor declarado; entre 15 y 58 registros por tabla.
- 13 consultas `SELECT` que responden preguntas del negocio; 8 de ellas combinan a la vez `JOIN`, `WHERE`, `GROUP BY` y `ORDER BY`.
- Agente en n8n: Chat Trigger → AI Agent (Gemini) + memoria + herramienta MySQL con usuario de solo lectura.

Uso de IA

Herramientas utilizadas: Claude (Anthropic) como asistente principal para explicar conceptos, revisar el script SQL y preparar el flujo de n8n; documentación oficial de MySQL 8, MySQL Workbench y n8n para verificar lo que la IA proponía; Google Gemini como modelo de lenguaje del agente.

Ejemplos de prompts que escribimos:

1. "*Explícame como si fuera principiante qué es una llave foránea y dame un ejemplo con una aduana que recibe contenedores de China.*"
2. "*Analiza este laboratorio y ayúdame a hacerlo: investigación, diagrama entidad-relación, base de datos en MySQL, y el agente en n8n.*"
3. "*Revisa mis CREATE TABLE: ¿qué restricciones CHECK, UNIQUE y DEFAULT le faltan a la tabla declaración?*"
4. "*Genera datos de prueba realistas: países reales, partidas del Sistema Armonizado reales y fechas lógicas (la inspección no puede ser antes de la declaración).*"
5. "*¿Cómo creo en MySQL un usuario que solo pueda hacer SELECT, para que el agente de n8n no pueda borrar datos?*"
6. "*Me da el error `ERROR 1142: DELETE command denied to user 'agente_lectura'`, ¿qué significa?*" (era justo la prueba de que el usuario de solo lectura funcionaba).

Qué aprendimos en el proceso:

- Que el diagrama se hace **antes** del SQL: al dibujar la relación declaración–producto descubrimos que era N:M y que necesitaba una tabla intermedia con llave compuesta.
- El orden importa: primero se crean e insertan las tablas "padre" (país, partida, bodega) y después las "hijas" (declaración, inventario); si no, la llave foránea falla.
- La normalización evita errores reales: al poner el porcentaje de arancel en la partida y no en cada producto, cambiarlo se hace en un solo lugar.
- La seguridad de un agente de IA **no puede depender solo del prompt**; debe estar en los permisos de la base de datos.
- Cada gestor tiene su sintaxis: al pasar el script de PostgreSQL a MySQL cambiamos `SERIAL` por `INT AUTO_INCREMENT`, `TO_CHAR` por `DATE_FORMAT` y `FILTER` por `SUM(CASE ...)`.
- Descubrimos que en MySQL un `REFERENCES` escrito junto a la columna **se ignora en silencio**: las llaves foráneas deben declararse con `FOREIGN KEY (...) REFERENCES ...`. Lo comprobamos consultando

`information_schema` (20 llaves foráneas creadas). Por eso siempre hay que ejecutar y probar el script, no solo confiar en lo que propone la IA.

Fuentes consultadas

1. Oracle. *MySQL 8.0 Reference Manual* — CREATE TABLE, FOREIGN KEY Constraints, CHECK Constraints, Account Management (CREATE USER, GRANT). https://dev.mysql.com/doc/refman/8.0/en/
2. PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/
3. SQLite. *SQLite Documentation — Foreign Key Support*. https://www.sqlite.org/foreignkeys.html
4. Oracle. *MySQL Workbench Manual*. https://dev.mysql.com/doc/workbench/en/
5. n8n. *Documentation — AI Agent node, MySQL node, Chat Trigger, Credentials*. https://docs.n8n.io/
6. Organización Mundial de Aduanas (OMA/WCO). *What is the Harmonized System (HS)?* https://www.wcoomd.org/en/topics/nomenclature/overview/what-is-the-harmonized-system.aspx
7. Elmasri, R. y Navathe, S. (2016). *Fundamentos de Sistemas de Bases de Datos* (7.ª ed.). Pearson.
8. Silberschatz, A., Korth, H. y Sudarshan, S. (2019). *Database System Concepts* (7.ª ed.). McGraw-Hill.
9. OWASP. *Top 10 for Large Language Model Applications — LLM01: Prompt Injection*. https://genai.owasp.org/